# Coding Challenge: AI Inbox Summary with Nylas

**Language:** TypeScript (Node.js)
**Target time:** ~4 focused hours.

You may use any coding agents or AI tools you have access to. We expect you to. Because of that, we are not testing typing speed; we're testing the decisions you make, the quality of what you ship, and whether you can explain and extend it.

## Background

People drown in email. We want a small app that connects to a user's mailbox, watches what comes in, and periodically emails them an AI-written digest of their inbox, so they can skip the firehose and read one good summary on whatever cadence they choose.

You'll build that app on top of the Nylas API, which gives you a single, provider-agnostic interface for authenticating a mailbox, reading messages, receiving real-time notifications, and sending email.

## Environment

We give you an **Ubuntu VM with a public IP and no firewall**. No setup, just a bare machine.

Install whatever you need. Because it's directly reachable from the internet, you can point the Nylas webhook straight at your app rather than using a dev tunnel. You will **sign up for your own (free) Nylas account and application**, create the API key and OAuth client credentials, and connect a mailbox to test with. You also **bring your own LLM API key** (any provider) for the summarizing "AI agent." Read all secrets from config/env. Never hard-code them.

## What to build

An app that does the following. **You choose the shape**: a CLI, a React app, a Next.js app, or a plain Node service. Note that two of the requirements (the webhook receiver and the scheduled send) need a component that can receive HTTP and run work on a schedule, so a browser-only client won't be enough on its own.

Pick an architecture that can actually satisfy all five points and explain it. A single full-stack app (e.g. Next.js) is perfectly acceptable; a monorepo is fine but not required.

## 1. Authenticate with Nylas

Let a user connect their mailbox via Nylas **hosted OAuth** (`/v3/connect/auth` → callback → exchange code for a **grant**). Persist the resulting `grantId` so the app can act on that mailbox across restarts. Handle the callback and at least the basic unhappy paths (denied consent, expired/invalid grant).

## 2. Read the inbox

Using the grant, read the user's inbox messages via Nylas (`GET /v3/grants/{grantId}/messages`). You'll need enough of each message (sender, subject, date, snippet/body, read state) to summarize it. Be deliberate about how much you pull and how you paginate — don't refetch the entire mailbox every time.

## 3. Periodic AI inbox summary, emailed out

On a schedule (see #5), generate a summary of the user's recent inbox activity using the **AI agent**, and **send it via Nylas** (`POST /v3/grants/{grantId}/messages/send`) to a **destination email address the user specifies** (which may differ from the connected mailbox).

The AI step should be a clean, testable seam: assembling the input from collected messages, calling the model, and parsing its output should be well-defined boundaries — not smeared through your scheduler. The summary should be genuinely useful (e.g. senders/threads that matter, asks awaiting a reply, deadlines), not just a list of subjects.

## 4. Ingest incoming mail via webhook

Register a Nylas **`message.created`** webhook pointed at your app. When a new email arrives, Nylas calls your endpoint; your app records that message and **accumulates incoming mail so the next scheduled summary covers everything since the last one.** Your handler must:

- complete the Nylas **challenge/verification** handshake,
- **verify the `x-nylas-signature` HMAC** before trusting a payload,
- acknowledge quickly (return `200` fast; do heavy work outside the request),
- and tolerate the realities of webhooks: **duplicate deliveries, out-of-order events, and truncated payloads** (refetch the full message when needed).

## 5. User-configurable cadence

Let the user choose how often they get a summary (e.g. hourly, every N hours, daily at a set time). Changing the cadence should take effect without code changes.

**Scheduling requirements (mechanism is your choice — `node-cron`, a job queue like BullMQ, a DB-backed poller, etc. — but these must hold):**

- Schedules **survive a process restart** (a chosen cadence isn't lost when the app reboots).
- Scheduling is **per-user/per-grant**, so different users can be on different cadences.
- A given summary window **fires once**. No duplicate emails if the process restarts, runs twice, or you later run more than one instance.

State clearly in your README which mechanism you chose and why, and how it meets the three points above.

# Deliverables

Submit:

**1. A git repository** (or zip with a clean git log) and a **README** with:

- How to install, configure (env vars), and run it — including how you exposed the webhook (HTTPS on the VM) and how to set up a Nylas app from scratch.
- The flow end to end: connect mailbox → collect incoming mail → scheduled summary → email sent.
- Your design decisions and tradeoffs, especially the scheduling/durability approach, how you collect and de-duplicate incoming mail, and how you structured the AI-agent seam to be testable/deterministic.
- What you'd do with more time, and anything you deliberately left out.

**2. A short demo video** (a few minutes, screen recording is fine) walking through the working app: connecting a mailbox, an incoming email being picked up via the webhook, and a generated summary arriving at the destination address.

## Constraints & notes

- TypeScript with reasonable type safety throughout. Avoid pervasive `any`.
- External dependencies (Nylas, the LLM) should sit behind clean interfaces so they can be swapped or faked. A live mailbox shouldn't be required to run your unit logic.
- Treat secrets and the webhook signature seriously; don't log full message bodies or tokens.
- Commit incrementally; we read the history.
- Coding agents are allowed and encouraged, but **you own every line and must be able to walk through and extend it live.** A clean, well-reasoned, smaller submission beats a sprawling one you can't defend.

If anything is ambiguous, make a reasonable assumption, state it in the README, and move on. Deciding under ambiguity is part of the exercise.